

Objectives:

1. To design a minimal computer system using Logisim focusing on fundamental components such as ALU, registers, instruction set architecture, MAR, MBR, IR, flags, control unit, CPU, and main memory.
2. To gain practical experience in computer architecture and digital logic design through hands-on implementation of essential components.
3. To understand the functionality and interaction of key components within a computer system, including data manipulation, instruction execution, and control flow.
4. To follow a sequential design approach, starting from basic building blocks and progressively integrating more complex components to form a functional computer system.
5. To test and verify the computer system's functionality through simulation using Logisim.

Introduction:

The evolution of computer systems has been one of the most transformative developments in human history, shaping the way we live, work, and communicate. At the core of every computer lies a complex interplay of digital logic components, meticulously designed to execute instructions and process data with remarkable efficiency and precision. Modern computer systems consist of various interconnected components that work together to execute instructions and process data. Understanding the functionality and interaction of these components is crucial for computer engineers and enthusiasts. This project aims to design and implement a simple CPU using Logisim, a digital logic simulator. The CPU will be capable of executing fundamental arithmetic and logic operations, including addition, subtraction, AND, OR, and NOT operations. This project serves as an educational tool to help students understand the basic principles of CPU architecture and digital logic design, with memory serving as the storage unit for both instructions and data.

The CPU architecture integrates a 19-bit ALU, comprising adders, logic gates, and multiplexers, enabling arithmetic (addition and subtraction) and logical (AND,

OR) operations. A control unit coordinates instruction fetching, decoding, and execution, ensuring precise operation sequencing.

Registers facilitate temporary data storage during computation, while memory modules store instructions and operands. The instruction set encompasses commands such as AND, ADD, STORE, ISZ, BSB, JMP, XOR, NAND, LOAD, and HALT, each triggering specific ALU operations and control unit actions. The CPU fetches instructions, decodes them into control signals, executes operations on input numbers fetched from registers or memory, and stores or outputs results accordingly, culminating in a comprehensive, functional CPU design.

The main memory module was implemented with a specified word size to store both instructions and data. It provides read and write access to the CPU, enabling the fetching of instructions and storage of data during computation.

Instruction Set:

A minimal instruction set was defined to support basic arithmetic (addition, subtraction), logical (AND, OR) operations, and looping. Each instruction was encoded into a binary format from hexadecimal format and interpreted by the CPU's instruction decoder.

Instruction Set Definitions:

1. AND (Logical AND)

- Performs a bitwise AND operation between two registers.
- Example: AND R1, R2 → Performs R1 AND R2 and stores the result in R1.

2. ADD (Addition)

- Adds the values of two registers or a register and an immediate value.
- Example: ADD R1, R2 → Adds the values in R1 and R2 and stores the result in R1.

3. STORE (STO)

- Stores a register value into memory.
- Example: STO R1, [500] → Stores the value of R1 into memory address 500.

4. Increment and Skip if Zero (ISZ)

- Increments a register and skips the next instruction if the result is zero.

- Example: ISZ R1 → Increments R1; if R1 becomes zero, the next instruction is skipped.
- 5. Branch Subroutine (BSB)**
 - Calls a subroutine by saving the return address.
 - Example: BSB 200 → Saves the current address and jumps to memory address 200.
- 6. Jump (JMP)**
 - Unconditionally jumps to a specified address.
 - Example: JMP 300 → Jumps to memory address 300.
- 7. LOAD (LOAD)**
 - Loads a value from memory into a register.
 - Example: LOAD R1, [100] → Loads the value from memory address 100 into R1.
- 8. HALT**
 - Stops the execution of the program.
 - Example: HALT → Halts the CPU until reset or restart.

TABLE FOR OPERATIONAL CODE FOR EACH INSTRUCTION:

OPERATION	OPCODE
AND	0000
ADD	0001
STO	0010
ISZ	0011
BSB	0100
BUN	0101
HALT	0110
LOAD	0111
OR	1000
SUB	1001
NAND	1010
XOR	1011

Format for address calculation:

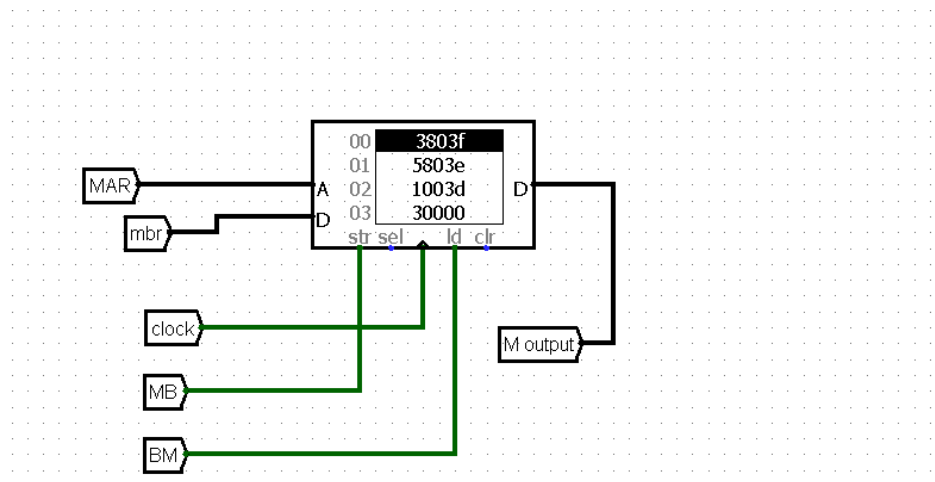
Here we have used the memory buffer register to split the memory address of 6-bit (LSB) and opcode of 4-bit in the MSB. The rest of the 9 bits of memory address is set to don't care.

For example, we want to store and operate ADD. We want to xor 9 and 5 in the memory address.

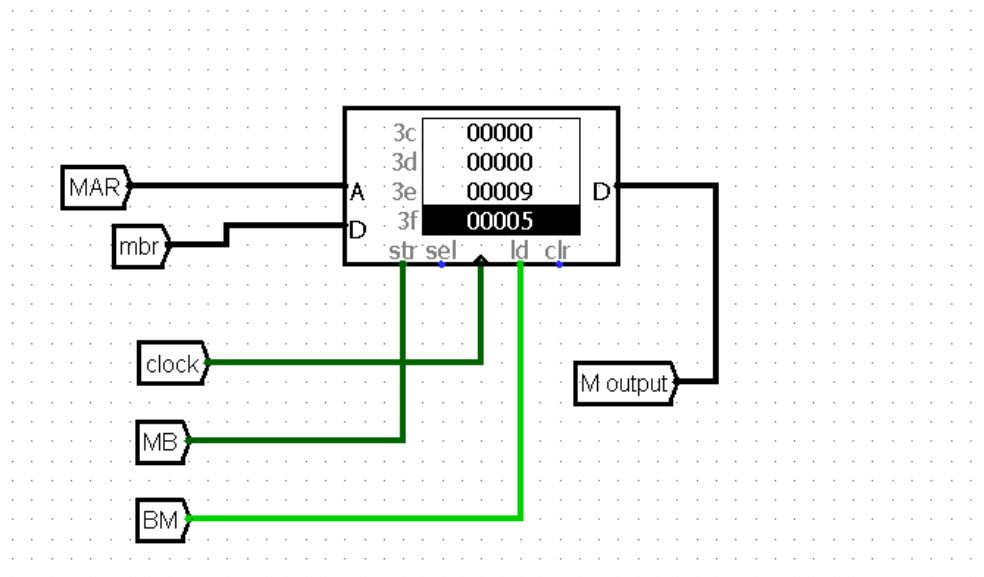
Location	Opcode 4 bits	Don't care 9 bits	Address 6 bits	HEX Address
0	0111(load)	00.....00	111111(63)	3803F
1	0001(XOR)	00.....00	111110(62)	5803E
2	0010(store)	00.....00	111101(61)	1003D
3	0110(halt)	00.....00	-	30000
...				
61				
62	-	-	-	00009
63	-	-	-	00005

Here the result will be 0000C in hexadecimal.

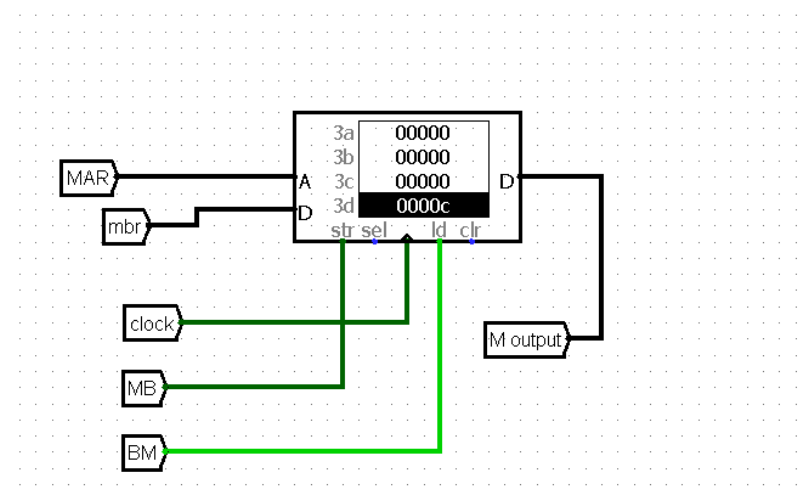
We will see the figure. Data load in the RAM.



Fetching 9 from the address 3803F and using the q11 operation (XOR) to fetch and xor 5 from the address 5803E. Then we will store the value in address 1003D



Now we can see that the result has been stored in the address 1003D and the result is $0C_{16}$.



In this instruction, we didn't use ALU directly. We had to use an external circuit to implement it. In the control unit, we had to manually implement it like the figure below:



Now in the main circuit, the AC and MBR are used as the input for the xor gate as usual for the ALU input.

The operation AND, OR, SUB, ADD was implemented using the ALU control using a 16 to 1 multiplexer. These opcodes were loaded into the instruction register and decoded to Alu's control of 2 bits like 00,01,10,11.

The other operations were implemented in the control unit using the time sequence with the fetch cycle and execution cycle.

Simulation :

The designed computer system was tested extensively through simulation in Logisim. Test programs were written to verify the correct execution of instructions and the proper functioning of the CPU and memory. Various scenarios were simulated to ensure the system's robustness and correctness.

Discussion:

In this project, I delved into the fascinating realm of computer architecture by constructing a 19-bit CPU from scratch using Logisim simulation. My endeavor aimed not only to grasp the intricacies of CPU design but also to gain practical insights into the functioning of fundamental computing components. Through meticulous planning, design iterations, and implementation, I embarked on a journey to construct a fully operational 19-bit CPU that encompasses essential components such as registers, ALU, control unit, and memory.

The foundation of my CPU design rested upon the careful selection and integration of individual components. Each component was meticulously crafted to fulfill its designated role within the CPU architecture. The registers served as crucial storage elements for temporary data manipulation, while the Arithmetic Logic Unit (ALU) undertook arithmetic and logical operations with precision. Additionally, the control unit orchestrated the flow of data and instructions, ensuring seamless execution of program sequences.

One of the pivotal aspects of my design was the incorporation of a 19-bit architecture. This decision necessitated meticulous attention to detail in component design and interconnectivity to accommodate the unique data width. By meticulously aligning the widths of registers, buses, and ALU operations, we ensured compatibility and consistency throughout the CPU architecture.

Conclusion:

The project successfully achieved its objectives of designing and implementing a minimal computer system using Logisim software. Through this endeavor, valuable insights into computer architecture and digital circuit design were gained. This project offers valuable insights into CPU architecture, instruction execution, and control flow management. By implementing basic arithmetic and logical operations, it lays a solid foundation for understanding complex CPU designs and computer architecture principles.

Furthermore, the simplicity of Logisim facilitates hands-on experimentation and comprehension of digital logic concepts. Future iterations could focus on optimizing performance and enhancing overall functionality for more advanced applications.